



CA - Experiment 2 - Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins

Maven: Basic Introduction

Maven is a build automation tool primarily used for Java projects. It simplifies the build process, manages project dependencies, and supports plugins for different tasks. It uses XML files (called `pom.xml`) for project configuration and dependency management.

Key Benefits of Maven:

- Dependency management (automates downloading and including libraries).
- Build automation (compiles code, runs tests, creates artifacts).
- Consistent project structure (standardizes how Java projects are set up).
- Integration with CI tools (like Jenkins).

Key Features of Maven:

- **Project Management:** Handles project dependencies, configurations, and builds.
- **Standard Directory Structure:** Encourages a consistent project layout across Java projects.
- **Build Automation:** Automates tasks such as compilation, testing, packaging, and deployment.
- **Dependency Management:** Downloads and manages libraries and dependencies from repositories (e.g., Maven Central).
- **Plugins:** Supports many plugins for various tasks like code analysis, packaging, and deploying.

Steps to Create a Maven Project in IntelliJ IDEA

1. Install Maven (if not already installed):

- Download Maven from the [official website](https://maven.apache.org/download.cgi).
- Set the `MAVEN_HOME` environment variable and update the system `PATH`.

2. Create a New Maven Project:

- Open IntelliJ IDEA.
- Go to `File > New > Project`.
- Select `Maven` from the project types.
- Choose `Create from Archetype` (optional) or proceed without.
- Set the project name and location, then click `Finish`.

3. Set Up the `pom.xml` File:

- The `pom.xml` file is where you define dependencies, plugins, and other configurations for your Maven project.
- Example of a basic `pom.xml`:

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0"
2     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```

3      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4      4.0.0.xsd">
5      <modelVersion>4.0.0</modelVersion>
6      <groupId>com.example</groupId>
7      <artifactId>simple-project</artifactId>
8      <version>1.0-SNAPSHOT</version>
9
10     <dependencies>
11         <!-- Add your dependencies here -->
12     </dependencies>
13
14 </project>
15

```

4. Add Dependencies for Selenium and TestNG:

- In the `pom.xml`, add Selenium and TestNG dependencies under the `<dependencies>` section.

```

1 <dependencies>
2   <dependency>
3     <groupId>org.seleniumhq.selenium</groupId>
4     <artifactId>selenium-java</artifactId>
5     <version>3.141.59</version>
6   </dependency>
7   <dependency>
8     <groupId>org.testng</groupId>
9     <artifactId>testng</artifactId>
10    <version>7.4.0</version>
11    <scope>test</scope>
12  </dependency>
13 </dependencies>
14

```

5. Create a Simple Website (HTML, CSS, and Logo):

- In the `src/main/resources` folder, create an `index.html` file, a `style.css` file, and place the `logo.png` image.

Example of a simple `index.html`:

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>My Simple Website</title>
7   <link rel="stylesheet" href="style.css">
8 </head>
9 <body>
10   <header>
11     
12   </header>
13   <h1>Welcome to My Simple Website</h1>
14 </body>
15 </html>
16

```

Example of a simple `style.css`:

```

1 body {
2   font-family: Arial, sans-serif;
3   background-color: #f4f4f4;
4   text-align: center;

```

```

5 }
6 header img {
7     width: 100px;
8 }
9

```

6. Upload the Website to GitHub:

- Initialize a Git repository in your project folder:

```

1 git init
2

```

- Add your files and commit them:

```

1 git add .
2 git commit -m "Initial commit"
3

```

- Create a GitHub repository and push the local project to GitHub:

```

1 git remote add origin <your-repository-url>
2 git push -u origin master
3

```

Deployment :

To deploy your Maven project to **GitHub Pages** using the `/docs` folder (**** having all files inside root folder/dir not recommended**), you can follow these simple steps. This method is easy and doesn't require switching branches—just use the `/docs` folder of your main branch.

Steps to Deploy to GitHub Pages Using `/docs` Folder

1. **Modify Maven Configuration to Copy Static Files to `/docs` Folder:** First, you need to ensure that Maven places your static files (`index.html`, `style.css`, `logo.png`) into the `/docs` folder instead of the `target` directory.

To do this, configure the **Maven Resources Plugin** in your `pom.xml` to copy the files directly into `/docs` :

```

1 <build>
2   <plugins>
3     <plugin>
4       <groupId>org.apache.maven.plugins</groupId>
5       <artifactId>maven-resources-plugin</artifactId>
6       <version>3.2.0</version>
7       <executions>
8         <execution>
9           <phase>prepare-package</phase> <!-- Before packaging -->
10          <goals>
11            <goal>copy-resources</goal>
12          </goals>
13          <configuration>
14            <outputDirectory>${project.basedir}/docs</outputDirectory> <!-- Deploy to
/docs folder -->
15            <resources>
16              <resource>
17                <directory>src/main/resources</directory>
18                <includes>
19                  <include>**/*</include> <!-- Copy all files in
src/main/resources -->
20                </includes>

```

```
21         </resource>
22     </resources>
23 </configuration>
24 </execution>
25 </executions>
26 </plugin>
27 </plugins>
28 </build>
29
```

In this configuration:

- The `maven-resources-plugin` copies all files from `src/main/resources` to the `/docs` folder in the root of your project (not the `target` directory).
- This is done during the `prepare-package` phase, just before Maven prepares the package.

2. **Build the Project:** Run the following Maven command to build your project and copy the resources to the `/docs` folder:

```
1 mvn clean install
2
```

After this, your `index.html`, `style.css`, and `logo.png` files should now be inside the `docs` folder in the root of your project.

3. **Push Changes to GitHub:** Now that the files are in the `/docs` folder, they are ready to be served by GitHub Pages.

Follow these steps:

- **Stage the changes** (the updated `/docs` folder):

```
1 git add docs/*
2 git commit -m "Deploy site to GitHub Pages"
3
```

- **Push to GitHub:**

```
1 git push origin master # Or the branch you are using, maybe 'main' in some cases
2
```

4. **Enable GitHub Pages:** After pushing to the main branch, follow these steps to enable GitHub Pages:

- Go to your GitHub repository.
- Navigate to **Settings > Pages** (on the left sidebar).
- Under the **Source** section, select the `main` branch and `/docs` folder as the source.
- Click **Save**.

5. **Access Your Website:** Your static website is now hosted on GitHub Pages! You can access it at:

```
1 https://<your-github-username>.github.io/<your-repository-name>/
2
```

Summary:

- **Maven Resources Plugin** is configured to copy static files (`index.html`, `style.css`, `logo.png`) into the `/docs` folder.
- **Build and push** the changes to your GitHub repository.
- Enable **GitHub Pages** using the `/docs` folder as the source.

By using the `/docs` folder in the main branch, you avoid the complexities of working with a separate `gh-pages` branch, and it's easier to maintain your website alongside your project.

Information about the /docs folder

The `/docs` folder does **not need to be manually created**. When you configure the Maven build to copy resources to the `/docs` folder, Maven will automatically create this folder when you run the `mvn clean install` command (if it doesn't already exist).

Here's a clearer breakdown:

1. The `/docs` Folder:

- This folder will be automatically created during the build process when Maven runs the `maven-resources-plugin`.
- Maven will copy the contents from `src/main/resources` into the `/docs` folder.

2. No Need for Manual Creation:

You don't need to manually create the `/docs` folder. Maven handles this automatically based on the configuration in your `pom.xml` file.

After the Build:

Once the build completes, check the root of your project directory, and you should see a `docs` folder with all the copied files inside it.

Summary:

The `/docs` folder is not something you need to create manually. Maven will generate it automatically and place the static files inside it based on your configuration in the `pom.xml` file. After building and pushing the changes to GitHub, you can enable GitHub Pages to serve the content from this folder.

Concise step-by-step guide to deploy your Maven project to GitHub Pages using the `/docs` folder, assuming you already have necessary files inside it.

Here's a concise step-by-step guide to deploy your Maven project to GitHub Pages using the `/docs` folder, assuming you already have your `index.html`, `style.css`, and other necessary files:

1. Prepare Your Maven Project

- Open your `pom.xml` file and configure the `maven-resources-plugin` to copy your static files (like `index.html`, `style.css`, etc.) into the `/docs` folder:

```

1 <build>
2   <plugins>
3     <plugin>
4       <groupId>org.apache.maven.plugins</groupId>
5       <artifactId>maven-resources-plugin</artifactId>
6       <version>3.2.0</version>
7       <executions>
8         <execution>
9           <phase>prepare-package</phase>
10          <goals>
11            <goal>copy-resources</goal>
12          </goals>
13          <configuration>
14            <outputDirectory>${project.basedir}/docs</outputDirectory>
15            <resources>
16              <resource>
17                <directory>src/main/resources</directory>
18                <includes>
19                  <include>**/*</include>
20                </includes>
21              </resource>

```

```
22         </resources>
23     </configuration>
24 </execution>
25 </executions>
26 </plugin>
27 </plugins>
28 </build>
29
```

This ensures that all your static files from `src/main/resources` are copied into the `docs` folder.

2. Build the Project

Run Maven to build the project:

```
1 mvn clean install
2
```

This will generate the `/docs` folder and place your static files there.

3. Push Changes to GitHub

- Stage and commit the changes (i.e., the new `/docs` folder with your static files):

```
1 git add docs/*
2 git commit -m "Deploy site to GitHub Pages"
3
```

- Push to your repository (assuming you're working with the `master` branch):

```
1 git push origin master
2
```

4. Enable GitHub Pages

- Go to your **GitHub repository**.
- Navigate to **Settings > Pages** (in the sidebar).
- Under the **Source** section, select:
 - Branch:** `main`
 - Folder:** `/docs`
- Click **Save**.

5. Access Your Website

Your website will now be live at:

```
1 https://<your-github-username>.github.io/<your-repository-name>/
2
```

Summary:

- Configure Maven:** Set up `maven-resources-plugin` in `pom.xml` to copy files to `/docs`.
- Build the Project:** Run `mvn clean install` to generate the `/docs` folder.
- Push to GitHub:** Stage and commit the `/docs` folder, then push to the `main` branch.
- Enable GitHub Pages:** Configure GitHub Pages to use the `/docs` folder.
- Access:** Your site will be hosted on GitHub Pages at `https://<your-username>.github.io/<repo-name>/`.

7. Write a Simple Selenium Test with TestNG:

- Create a new Java class `WebPageTest.java` in the `src/test/java` directory.

Example of a simple TestNG test using Selenium:

```
1 package org.test;
2
3 import org.openqa.selenium.WebDriver;
4 import org.openqa.selenium.chrome.ChromeDriver;
5 import org.testng.Assert;
6 import org.testng.annotations.AfterTest;
7 import org.testng.annotations.BeforeTest;
8 import org.testng.annotations.Test;
9
10 import static org.testng.Assert.assertTrue;
11
12 public class WebpageTest {
13     private static WebDriver driver;
14
15     @BeforeTest
16     public void openBrowser() throws InterruptedException {
17         driver = new ChromeDriver();
18         driver.manage().window().maximize();
19         Thread.sleep(2000);
20         driver.get("https://sauravsarkar-codersarcade.github.io/CA-MVN/"); // "Note: You should
21         use your GITHUB-URL here...!!!"
22     }
23
24     @Test
25     public void titleValidationTest(){
26         String actualTitle = driver.getTitle();
27         String expectedTitle = "Tripillar Solutions";
28         Assert.assertEquals(actualTitle, expectedTitle);
29         assertTrue(true, "Title should contain 'Tripillar'");
30     }
31
32     @AfterTest
33     public void closeBrowser() throws InterruptedException {
34         Thread.sleep(1000);
35         driver.quit();
36     }
37 }
38
```

8. Run the Test:

- In IntelliJ, right-click the `WebPageTest` class and select `Run 'WebPageTest'`.
- The test will launch Chrome, open the webpage, and validate the title.

Summary of Steps:

1. Set up Maven project and configure `pom.xml`.
2. Create a simple website with HTML, CSS, and a logo image.
3. Upload the project to GitHub.
4. Write and run a Selenium test with TestNG to validate the webpage title.

 **Testing** the title of your website using **Selenium**, **Java**, and **TestNG**:

To test the title of your website using **Selenium**, **Java**, and **TestNG**, follow these steps. This will include the installation of necessary dependencies, creating test scripts, and running tests.

1. Set Up Selenium and TestNG Dependencies

In your Maven project, add the necessary dependencies for **Selenium WebDriver** and **TestNG** to the `pom.xml` file: (i) **Skip if already added...!!**

```
1 <dependencies>
2   <!-- Selenium WebDriver dependency -->
3   <dependency>
4     <groupId>org.seleniumhq.selenium</groupId>
5     <artifactId>selenium-java</artifactId>
6     <version>4.8.0</version> <!-- Ensure this is the latest version -->
7   </dependency>
8
9   <!-- TestNG dependency -->
10  <dependency>
11    <groupId>org.testng</groupId>
12    <artifactId>testng</artifactId>
13    <version>7.7.0</version> <!-- Ensure this is the latest version -->
14    <scope>test</scope>
15  </dependency>
16 </dependencies>
17
```

- **Selenium WebDriver:** This is used for browser automation.
- **TestNG:** This is a testing framework used to run Selenium tests.

2. Create Selenium Test Class Using TestNG

Next, create a test class in your `src/test/java` directory. You can name it `WebsiteTitleTest.java`.

Sample Code for Testing Website Title:

```
1 package org.test;
2
3 import org.openqa.selenium.WebDriver;
4 import org.openqa.selenium.chrome.ChromeDriver;
5 import org.testng.Assert;
6 import org.testng.annotations.AfterTest;
7 import org.testng.annotations.BeforeTest;
8 import org.testng.annotations.Test;
9
10 import static org.testng.Assert.assertTrue;
11
12 public class WebpageTest {
13     private static WebDriver driver;
14
15     @BeforeTest
16     public void openBrowser() throws InterruptedException {
17         driver = new ChromeDriver();
18         driver.manage().window().maximize();
19         Thread.sleep(2000);
20         driver.get("https://sauravsarkar-codersarcade.github.io/CA-MVN/"); // "Note: You should use
your GITHUB-URL here...!!!"
21     }
22
23     @Test
24     public void titleValidationTest(){
25         String actualTitle = driver.getTitle();
```

```
26     String expectedTitle = "Tripillar Solutions"; // "Replace with Your HTML WebPage Title"
27     Assert.assertEquals(actualTitle, expectedTitle);
28 }
29
30 @AfterTest
31 public void closeBrowser() throws InterruptedException {
32     Thread.sleep(1000);
33     driver.quit();
34 }
35
36 }
37
```

Code Explanation (Step-by-Step)

📌 Package Declaration

- `package org.test;`
 - Defines the package name as `org.test` (helps organize code).

📌 Imports Required Libraries

- `import org.openqa.selenium.WebDriver;` → Selenium WebDriver interface.
- `import org.openqa.selenium.chrome.ChromeDriver;` → Controls Google Chrome.
- `import org.testng.Assert;` → Provides assertion methods for validation.
- `import org.testng.annotations.*;` → TestNG annotations for test execution.
- `import static org.testng.Assert.assertTrue;` → Allows direct usage of `assertTrue()`.

📌 Class Declaration

- `public class WebpageTest {}` → Defines a **test class** for webpage testing.

📌 WebDriver Declaration

- `private static WebDriver driver;`
 - Declares a static WebDriver instance for browser control.

1 @BeforeTest - Setup Method

```
1 @BeforeTest
2 public void openBrowser() throws InterruptedException {
3
```

- Runs **before any test case** in this class.
- Initializes `ChromeDriver()` (opens Chrome).
- Maximizes the browser window.
- Waits for **2 seconds** (`Thread.sleep(2000)`).
- Navigates to `"https://sauravsarkar-codersarcade.github.io/CA-MVN/"`.

2 @Test - Title Validation Test

```
1 @Test
2 public void titleValidationTest() {
3
```

- Retrieves the **actual title** of the web page using `driver.getTitle()`.
- Defines the **expected title** as `"Tripillar Solutions"`.

- Uses `Assert.assertEquals(actualTitle, expectedTitle);`
 - **Passes** if the title matches `"Tripillar Solutions"`.
 - **Fails** if the title is different.

3 @AfterTest - Cleanup Method

```
1 @AfterTest
2 public void closeBrowser() throws InterruptedException {
3
```

- Runs **after all test cases** in this class.
- Waits for **1 second** (`Thread.sleep(1000)`).
- Closes the browser using `driver.quit();`.

Key Takeaways

- ✓ **Selenium WebDriver** automates browser interaction.
- ✓ **TestNG Annotations** manage test setup (`@BeforeTest`), execution (`@Test`), and teardown (`@AfterTest`).
- ✓ **Assertions** (`Assert.assertEquals()`) validate webpage content.

3. Run the Test Using TestNG

Option 1: Run TestNG from IntelliJ IDEA

1. Right-click the `WebpageTest.java` file.
2. Select **Run WebpageTest**.
 - IntelliJ IDEA will execute the TestNG test and show the results in the output console.

Option 2: Run TestNG via Command Line

If you want to run the tests from the command line, use the following Maven command:

```
1 mvn test
2
```

This will trigger the TestNG tests and execute all the tests in the `src/test/java` directory.

4. Verify the Test Results

After running the test, TestNG will output the results:

- If the website title matches the expected title, the test will pass.
- If the title is incorrect, the test will fail, and you will get an assertion error with a message indicating that the title doesn't match.

Summary of Steps:

1. **Add Dependencies:**
 - Add `selenium-java`, `testng`, and `webdrivermanager` dependencies in `pom.xml`.
2. **Write Test Class:**
 - Create a class `WebpageTest.java` with methods to launch the browser, navigate to the website, and verify the title.
3. **Run the Tests:**

- Use IntelliJ IDEA or Maven to run the tests and check the output for validation.

4. Verify Results:

- Ensure that the actual title matches the expected title. If they match, the test passes; if not, it fails.

This approach tests whether the website is correctly deployed and whether the title is as expected on your GitHub Pages site.

 Use of **maven-jar-plugin** to **package** the project into a **jar file**. | Showing how to **run a main class** and show simple **output**:

Great! You're on the right track. Here's a quick summary of what we have covered and additional steps you can take to demonstrate the use of **maven-jar-plugin** and how to package the project into a JAR file, including running a **main** class and showing simple output.

Summary of What We've Done So Far:

1. Website Deployment:

- You've deployed your simple HTML, CSS, and assets (like logo.png) to **GitHub Pages** using Maven.
- You've created a **Selenium Test** to validate the website title using **TestNG** and ran the test to ensure the website is functioning as expected.

2. Next Steps:

- We can show how to use the **maven-jar-plugin** to create a runnable JAR file.
- Demonstrate running a **main class** inside this JAR to produce a simple output.

Steps to Package the Project as a JAR and Run a Main Class

1. Add **maven-jar-plugin** to **pom.xml**:

To package your Maven project as a JAR file and specify the **main** class, we need to configure the **maven-jar-plugin** in the **pom.xml**.

Add the following configuration to your **pom.xml**:

```
1 <build>
2   <plugins>
3     <!-- Maven JAR Plugin -->
4     <plugin>
5       <groupId>org.apache.maven.plugins</groupId>
6       <artifactId>maven-jar-plugin</artifactId>
7       <version>3.2.0</version>
8       <configuration>
9         <!-- Specify the main class to be executed -->
10        <archive>
11          <manifestEntries>
12            <Main-Class>com.example.MainClass</Main-Class> <!-- Replace with your main
class path -->
13          </manifestEntries>
14        </archive>
15      </configuration>
16    </plugin>
17  </plugins>
18 </build>
19
```

This will tell Maven to include the **Main-Class** in the JAR manifest and specify the main class that should be executed when the JAR is run.

2. Create a Main Class:

In your `src/main/java` directory, create a class with a `main` method. For example, create a `MainClass.java` under `com.example`:

```
1 package com.example;
2
3 public class MainClass {
4     public static void main(String[] args) {
5         System.out.println("Hello, this is a simple output from the main class!");
6     }
7 }
8
```

This class contains a simple `main` method that prints output when run.

3. Package the Project into a JAR:

After configuring the plugin and creating the `MainClass`, run the following Maven command to build the project and package it into a JAR file:

```
1 mvn clean package
2
```

This will clean any previous builds, compile the source code, and package it into a JAR file located in the `target` directory (e.g., `target/your-project-name.jar`).

4. Run the JAR File:

Once the JAR is created, you can run it with the following command:

```
1 java -jar target/your-project-name.jar
2
```

This will execute the `main` method from your `MainClass` and print the message:

```
1 Hello, this is a simple output from the main class!
2
```

Summary:

1. **Add maven-jar-plugin:** Configure the plugin in `pom.xml` to specify the main class.
2. **Create Main Class:** Write a simple `MainClass` with a `main` method that outputs a message.
3. **Package with Maven:** Run `mvn clean package` to package the project into a JAR file.
4. **Run the JAR:** Use `java -jar target/your-project-name.jar` to run the packaged JAR and print the output.

Final Thoughts:

- By showing this process, you demonstrate how Maven is used to automate the build, test, deployment, and packaging of projects.
- The **maven-jar-plugin** allows you to easily create runnable JAR files, and running the `main` class from the JAR file is a common way to run Java applications.

This provides a good conclusion to the topic, demonstrating not only the deployment and testing of the website but also the power of Maven for packaging and managing Java projects.

 **Important Note :**